SCÉNÁŘ PŘEDNESU · KIV/PTE POKROČILÉ TESTOVÁNÍ · ZČU FAV

Coordinated omission

a pasti průměrných časů

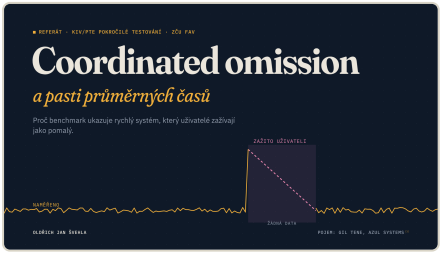
OLDŘICH JAN ŠVEHLA

15 SLIDŮ · PLÁN 9:30

ZVÝRAZNĚNÍ
Klíčová věta, kterou je dobré říct skoro doslova.

KLIK
▶ KLIK posune slide o krok (šipka → nebo mezerník).

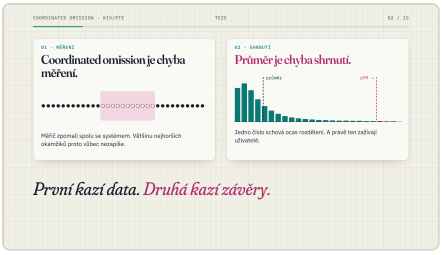
ČAS
Kumulativní plán; v prezentaci ukazuje režim řečníka (klávesa P) i časovač.



01 0:00–0:30 · ~30 s

Úvod

Dobrý den. Když děláme benchmark nebo zátěžový test, jednou z hlavních metrik je doba odezvy. Dnes vám ukážu, **proč naměřená doba odezvy často lže, i když nikdo nic nefalšuje**. Příčiny jsou dvě. **První je coordinated omission**, česky koordinované vynechání. Ten pojem zavedl Gil Tene ze společnosti Azul Systems. **Druhou jsou průměry**, a obecně to, jak z tisíců měření uděláme jedno číslo. Graf na pozadí je celý problém v malém: měřič zapsal jediný pomalý vzorek a během výpadku pak nezapsal nic. Růžově je to, co mezitím zažili uživatelé.

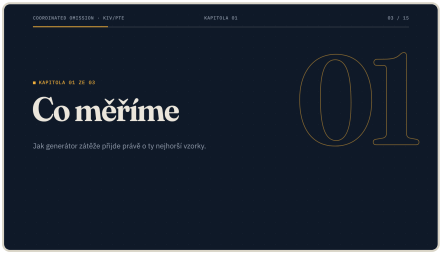


02 0:30–1:15 · ~45 s

Teze

Hned na začátek hlavní myšlenka. **Coordinated omission je chyba v tom, CO naměříte**. Měřicí nástroj zpomalí přesně ve chvíli, kdy zpomalí systém, a většina nejhorších okamžiků se do dat vůbec nedostane. To jsou ty prázdné kroužky vlevo. **Průměr je chyba v tom, JAK naměřené shrnete**. I s dokonalými daty jedno číslo schová ocas rozdělení, tedy ty nejpomalejší odpovědi vpravo. A právě ty si uživatelé pamatují.

▶ KLIK Takže: **první chyba kazí data, druhá kazí závěry**. A nejhorší je, když se potkají obě najednou.



03 1:15–1:20 · ~5 s

Předěl: Co měříme

Začněme tím, co vlastně měříme.



04 1:20–2:30 · ~70 s

Uzavřená smyčka

Většina klasických nástrojů pro zátěžové testy **funguje v takzvané uzavřené smyčce**. Tak funguje i běžná Thread Group v JMeteru. Každé vlákno pošle požadavek, počká na odpověď, zapíše, jak dlouho to trvalo, a teprve potom pošle další. Horní řada ukazuje, kdy jsme chtěli požadavky posílat, třeba sto za sekundu. Dokud server odpovídá rychle, spodní řada plán věrně kopíruje.

▶ KLIK Teď se server na chvíli zasekne, třeba kvůli garbage collectoru, zámku v databázi nebo přetížené síti. Požadavek, který zrovna běží, čeká, a generátor čeká s ním. Až odpověď dorazí, zapíše se jeden pomalý vzorek.

▶ KLIK A tady je ten problém. Všechny tyhle požadavky měly během zaseknutí odejít, ale generátor je neposlal, protože čekal. **Kdo je neposlal, ten je ani nezměřil**. Z celého výpadku tak v datech zůstane **jediný pomalý vzorek mezi stovkami rychlých**.



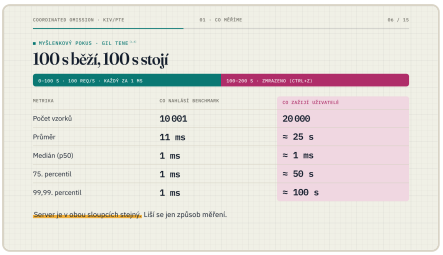
05 2:30–3:15 · ~45 s

Definice

Gil Tene tomu říká coordinated omission. Měřený systém a měřicí nástroj se nevědomky sladí: **když systém zpomalí, měřič zpomalí s ním a přestane se ptát**.

▶ KLIK Je to jako anketa spokojenosti, kterou dělá někdo, kdo se ptá jen lidí odcházejících od přepážky. Když se přepážka na hodinu zavře, nikdo neodchází, a tedy si nikdo nestěžuje. Fronta venku do statistiky vůbec nevstoupí.

▶ KLIK Výsledek pak popisuje hlavně dobu obsluhy, anglicky service time, tedy jak dlouho trvalo vyřídit požadavek, když na něj přišla řada. **Uživatel ale zažívá dobu odezvy, response time: čekání ve frontě plus obsluhu**.



06 3:15–4:20 · ~65 s

Myšlenkový pokus: 100 s běží, 100 s stojí

Jak velký to má dopad? Tene ve své přednášce používá myšlenkový pokus. Systém sto sekund odpovídá za jednu milisekundu při sto požadavcích za sekundu. Pak ho na sto sekund zmrazíme, třeba přes Ctrl+Z. Generátor v uzavřené smyčce nasbírá deset tisíc rychlých vzorků a jeden vzorek o délce sto sekund. Průměr vyjde necelých jedenáct milisekund a **i 99,99. percentil ukazuje jednu milisekundu**. Skvělý systém.

▶ KLIK Kdyby ale uživatelé chodili dál podle plánu, během zmrazení by přišlo dalších deset tisíc požadavků a čekaly by v průměru padesát sekund. **Průměr přes dvě stě sekund je pak asi pětadvacet sekund**, 75. percentil padesát a 99,99. percentil sto sekund. Medián je pořád kolem milisekundy, takže ani ten nepomůže.

▶ KLIK **Server je v obou sloupcích stejný. Liší se jen způsob měření**.



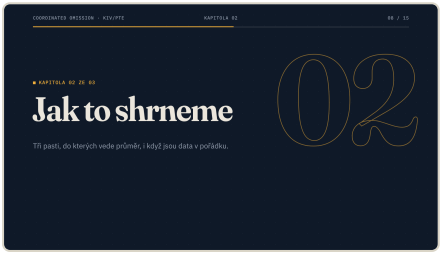
07 4:20–4:50 · ~30 s

wrk2: 200×

Není to jen teorie. Tene napsal nástroj wrk2, který měří správně a umí vypsat i nekorigovaná čísla, takže jde obojí porovnat na jednom běhu. V příkladu z jeho dokumentace se server během třicetisekundového testu na 1,4 sekundy pozastaví. **Nekorigovaný 99. percentil vyjde šest milisekund**.

▶ KLIK **Korigovaný 1 270 milisekund, zhruba dvěstěkrát víc**.

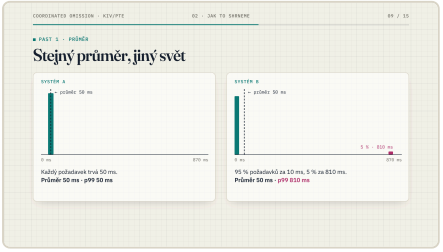
▶ KLIK Rozdíl je jen v tom, že korigovaná verze **počítá čas od chvíle, kdy požadavek MĚL odejít**, a ne kdy skutečně odešel.



08 4:50–5:00 · ~10 s

Předěl: Jak to shrneme

Teď předpokládejme, že měříme správně a data máme v pořádku. Pořád je musíme nějak shrnout do čísla, které někomu ukážeme. A tady čekají tři pasti.

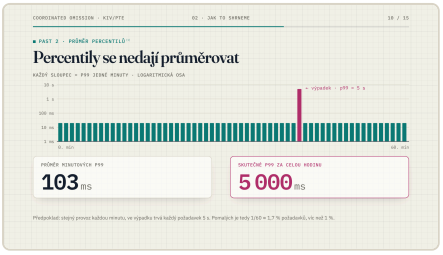


09 5:00–5:45 · ~45 s

Past 1: stejný průměr, jiný svět

První past: průměr nic neříká o tvaru rozdělení. Systém A odpoví na každý požadavek za padesát milisekund. Systém B odpoví na 95 procent požadavků za deset milisekund a na zbylých pět procent za 810 milisekund. Spočítejte si to: 0,95 krát 10 plus 0,05 krát 810 je přesně padesát. **Oba mají stejný průměr**.

▶ KLIK Ale **99. percentil systému B je 810 milisekund**, šestnáctkrát víc než u systému A. Latence v reálných systémech obvykle nemá tvar Gaussova zvonu. Mívá dlouhý ocas doprava a často víc vrcholů, takže průměr ani směrodatná odchylka její tvar nepopíšou. **Proto se hlásí percentily a maximum, ne jedno číslo**.



10 5:45–6:35 · ~50 s

Past 2: percentily se nedají průměrovat

Druhou past najdete skoro v každém dashboardu. Monitoring spočítá 99. percentil za každou minutu a pro hodinový pohled ho pak zprůměruje. Tady je hodina, kde 59 minut má p99 dvacet milisekund a jedna minuta je výpadek, kdy každý požadavek trvá pět sekund. **Pozor, osa je logaritmická**, jinak by dvacet milisekund nebylo vůbec vidět. Průměr minutových p99 vyjde 103 milisekund, mírné zhoršení.

▶ KLIK Jenže při rovnoměrném provozu tvoří ta jedna minuta šedesátinu všech požadavků, asi 1,7 procenta. To je víc než jedno procento, takže **skutečný 99. percentil za hodinu je pět sekund**. Tene to říká přímo: **percentily průměrovat nejde**. Když agregujete přes čas nebo servery, **slučujte celé histogramy** a percentil spočítejte až z nich.



11 6:35–7:25 · ~50 s

Past 3: fan-out (The Tail at Scale)

Třetí past: díváme se na jeden server, ale uživatel čeká na mnoho. Jeff Dean a Luiz Barroso z Googlu to popsali v článku The Tail at Scale. Představte si server, který obvykle odpoví za deset milisekund, ale jeho 99. percentil je jedna sekunda. Jeden požadavek ze sta tedy trvá sekundu nebo déle. Pro jednotlivé volání je to jedno procento, ocas, který v průměru zmizí.

▶ KLIK Jenže jeden dotaz uživatele se ve velkém systému často rozpadne na desítky až stovky paralelních volání a odpověď je hotová, až když dorazí ta nejpomalejší. Když dotaz čeká na sto takových serverů, pravděpodobnost, že aspoň jeden bude pomalý, je **jedna minus 0,99 na stou, tedy 63 procent**.

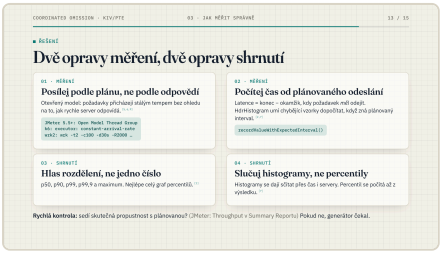
▶ KLIK Co je na jednom serveru vzácný ocas, je pro uživatele běžný zážitek. Proto u latence nestačí sledovat průměr ani medián. **Rozhoduje ocas**.



12 7:25–7:30 · ~5 s

Předěl: Jak měřit správně

Co s tím tedy dělat?



13 7:30–8:45 · ~75 s

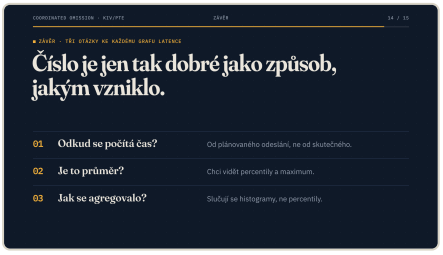
Řešení: dvě opravy měření, dvě opravy shrnutí

Co s tím? Dvě opravy měření a dvě opravy shrnutí. První: **zátěž posílat podle plánu, ne podle odpovědí**. Tomu se říká otevřený model a popsali ho už v roce 2006 Schroeder, Wierman a Harchol-Balter. V JMeteru ho od verze 5.5 dělá **Open Model Thread Group**, která plánuje příchody požadavků místo počtu vláken. Podobně to umí k6 nebo wrk2.

▶ KLIK Druhá: **čas počítat od okamžiku, kdy požadavek MĚL odejít**. HdrHistogram umí chybějící vzorky dopočítat, když zná plánovaný interval.

▶ KLIK Třetí: místo jednoho čísla **hlásit rozdělení, tedy percentily a maximum**.

▶ KLIK Čtvrtá: při agregaci přes čas nebo servery **slučovat histogramy, ne percentily**. A nejrychlejší kontrola, kterou můžete udělat hned: **sedí skutečná propustnost testu s plánovanou**? Pokud generátor poslal méně požadavků, než měl, čekal.



14 8:45–9:20 · ~35 s

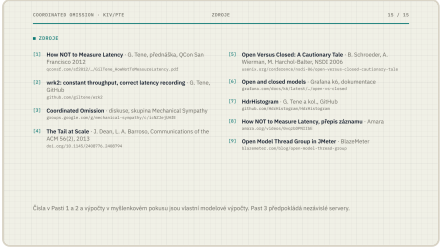
Závěr

Na závěr. Ve všech příkladech s měřením byl server pokaždé stejný. Rozdíl dělal jen způsob, jak jsme čísla získali a jak jsme je shrnuli. **Číslo je jen tak dobré jako způsob, jakým vzniklo**. Až příště uvidíte graf latence, položte si tři otázky.

▶ KLIK Odkud se počítá čas? Od plánovaného odeslání, nebo až od skutečného?

▶ KLIK Je to průměr? Pokud ano, chtějte percentily a maximum.

▶ KLIK A jak se data agregovala? Pokud se průměrovaly percentily, číslo neplatí.



15 9:20–9:30 · ~10 s

Zdroje a poděkování

Zdroje najdete tady, celá prezentace je online i s odkazy. Děkuji za pozornost a rád odpovím na otázky.

PŘÍLOHA

Co si nastudovat

Tahle část se neříká nahlas. Je to podklad, abys rozuměl tomu, co stojí za slidy, a zvládl otázky po referátu.

1 · Pojmy v jedné větě

Doba odezvy `response time`	Čas od chvíle, kdy klient požadavek odeslal (správně: *měl* odeslat), do přijetí odpovědi. Zahrnuje **čekání ve frontě i obsluhu**.
Doba obsluhy `service time`	Čas, kdy server požadavek skutečně zpracovává. Tohle měří generátor zasažený coordinated omission.
Propustnost `throughput`	Počet dokončených požadavků za jednotku času (req/s).
Percentil pXX	Hodnota, pod kterou leží XX % měření. p50 = medián; p99 = jen 1 % požadavků je pomalejší.
Ocas rozdělení `tail`	Nejpomalejší odpovědi, typicky nad p99. Průměr a medián o něm nic neřeknou.
Uzavřený model `closed`	Pevný počet klientů (vláken); každý pošle další požadavek až po odpovědi, případně po pauze (think time). **Tempo příchodů závisí na serveru.**
Otevřený model `open`	Požadavky přicházejí podle plánu (rychlostí λ) **nezávisle na tom, jak rychle server odpovídá**. Odpovídá veřejnému webu nebo API.
Coordinated omission	Systematická chyba měření: když systém zpomalí, generátor přestane posílat a měřit požadavky, takže **vynechá právě ty nejhorší vzorky**. Pojem zavedl Gil Tene (Azul Systems).
HdrHistogram	Histogram s velkým rozsahem hodnot a nastavitelnou přesností. Zápis je levný a **histogramy se dají sčítat**, takže jdou slučovat přes čas i servery.

2 · Proč generátor „zpomalí s ním“

Pro uzavřený systém platí **zákon interaktivní doby odezvy** z operační analýzy (Lazowska a kol., *Quantitative System Performance*, kap. 3):

$$X = N / (R + Z)$$

kde **X** je propustnost, **N** počet klientů (vláken), **R** doba odezvy a **Z** pauza mezi požadavky. Příklad: 200 vláken, Z = 1 s, R = 5 ms → X ≈ 199 req/s. Když se server zasekne a R vyroste, **X automaticky klesne**: generátor posílá méně požadavků přesně ve chvíli, kdy by měl systém zatěžovat nejvíc. To je mechanismus coordinated omission v jednom vzorci.

Příbuzný je **Littleův zákon** L = λ · W: průměrný počet požadavků v systému = rychlost příchodů × průměrná doba v systému. Platí pro otevřený i uzavřený systém a hodí se na rychlou kontrolu, jestli čísla z testu dávají smysl.

3 · Jak funguje korekce

- **wrk2** s parametrem `-R` drží konstantní propustnost. Každý požadavek má plánovaný čas odeslání a **latence se počítá od plánu**, ne od skutečného odeslání. Volbou `--u_latency` vypíše i nekorigovaná čísla pro srovnání.
- **HdrHistogram** `recordValueWithExpectedInterval(hodnota, interval)`: když je hodnota větší než očekávaný interval mezi požadavky, doplní navíc hodnoty *hodnota − interval*, *hodnota − 2·interval*, … dokud neklesnou pod interval. Příklad z myšlenkového pokusu: interval 10 ms (100 req/s), naměřeno 100 s → doplní se 9 999 hodnot od 99,99 s dolů po 10 ms. To jsou přesně požadavky, které generátor během výpadku nevyslal.
- Korekce je **odhad**, který předpokládá rovnoměrné příchody. Lepší je měřit rovnou v otevřeném modelu.

4 · Výpočty ze slidů

Myšlenkový pokus, benchmark	(10 000 × 1 ms + 1 × 100 000 ms) / 10 001 ≈ 11,0 ms. Tene v přednášce uvádí „10,9 ms“, přesně je to 10,999 ms.
Myšlenkový pokus, uživatelé	10 000 požadavků po 1 ms + 10 000 požadavků čekajících 100 s, 99,99 s, … 0,01 s (průměr ≈ 50 s) → (10 s + 10 000 × 50,005 s) / 20 000 ≈ 25 s. p75 = 15 000. hodnota ≈ 50 s, p99,99 ≈ 100 s.
wrk2	1 270 ms / 6,04 ms ≈ 210 → README uvádí „200x“.
Past 1	0,95 × 10 ms + 0,05 × 810 ms = 9,5 + 40,5 = 50 ms. p99 systému B leží v horních 5 %, tedy 810 ms.
Past 2	(59 × 20 ms + 5 000 ms) / 60 = 6 180 / 60 = 103 ms. Pomalá minuta = 1/60 ≈ 1,67 % požadavků > 1 % → skutečné p99 = 5 s.
Past 3	P(aspoň jeden pomalý z n) = $1 - 0{,}99^n$. n = 10 → 9,6 %, n = 100 → 63,4 %. Předpoklad: servery jsou pomalé nezávisle na sobě.

5 · Možné otázky po referátu

Q · Je uzavřený model vždycky špatně?

> Ne. Pokud má reálný systém pevný počet klientů, kteří na odpověď opravdu čekají (dávkové zpracování, pevný počet terminálů), uzavřený model ho popisuje věrně. Problém je, když reálné příchody na rychlosti serveru nezávisí, jako u webu nebo veřejného API. Schroeder, Wierman a Harchol-Balter ukazují, že **volba modelu výrazně mění výsledky**, takže ji musí určovat realita, ne pohodlí nástroje.

Q · Nestačí v JMeteru přidat Constant Throughput Timer?

> Ne. Timer jen brzdí vlákna na cílovou propustnost. Když server stojí, vlákna dál čekají na odpovědi a nové požadavky neodcházejí. Víc vláken problém zmírní, ale neodstraní. Řešením je **Open Model Thread Group** (JMeter 5.5+) a kontrola skutečné propustnosti proti plánu.

Q · Proč nehlásit prostě maximum?

> Maximum je důležité a coordinated omission ho obvykle neschová, protože ten jeden pomalý vzorek se zapíše. Neřekne ale, *jak často* se to děje: jeden výpadek za den, nebo polovina požadavků? Proto **percentily a maximum dohromady**.

Q · Jak dostanu p99 za hodinu, když mám uložené jen minutové p99?

> Přesně nijak, z percentilů se zpět rozdělení nesložíš. Je potřeba ukládat **histogramy** (HdrHistogram, případně přibližné struktury jako t-digest), sloučit je a percentil spočítat až z výsledku.

Q · Proč se v myšlenkovém pokusu nezměnil medián?

> Polovina všech požadavků (prvních 100 s) je rychlá, takže medián leží přesně na hranici a zůstane na 1 ms. Medián o horší polovině neříká vůbec nic.

Q · Jak poznám coordinated omission ve výsledcích?

> Nejrychleji: **skutečná propustnost je nižší než plánovaná** (v JMeteru Throughput v Summary Reportu). Dále „díry“ v časové ose vzorků v době, kdy systém zpomalil.

Q · Dá se s ocasem u fan-outu něco dělat?

> Dean a Barroso navrhují mimo jiné **hedged requests**: když odpověď nepřijde do očekávaného p95, pošle se stejný požadavek ještě jiné replice a použije se ta rychlejší odpověď. Zátěž to zvýší jen asi o 5 % a ocas výrazně zkrátí.

Q · Týká se to jen nástrojů na zátěžové testy?

> Ne. Vznikne všude, kde měřicí smyčka čeká na měřený systém: v mikrobenchmarcích, v klientovi, který si měří vlastní volání, nebo v monitoringu, který vidí jen dokončené požadavky.

---

6 · Doporučené studium

1. **Gil Tene: How NOT to Measure Latency** (přednáška, slidy z QCon SF 2012, přepis na Amara). Základ celého referátu: coordinated omission, percentily, HdrHistogram.
   `qconsf.com/sf2012 · amara.org/videos/0vqzbDPNIIbE`
2. **README nástroje wrk2**, část o coordinated omission a příklad s pozastavením serveru přes Ctrl+Z.
   `github.com/giltene/wrk2`
3. **J. Dean, L. A. Barroso: The Tail at Scale** (Communications of the ACM, 2013). Krátký článek o ocasu latence ve velkých systémech a technikách proti němu.
   `doi.org/10.1145/2408776.2408794`
4. **B. Schroeder, A. Wierman, M. Harchol-Balter: Open Versus Closed: A Cautionary Tale** (NSDI 2006). Kdy použít otevřený a kdy uzavřený model.
   `usenix.org/conference/nsdi-06`
5. **Dokumentace k6: Open and closed models**, stručné vysvětlení s příklady executorů.
   `grafana.com/docs/k6`
6. **BlazeMeter: Open Model Thread Group**, jak nastavit otevřený model v JMeteru.
   `blazemeter.com/blog/open-model-thread-group`
7. **README knihovny HdrHistogram**, záznam s očekávaným intervalem a slučování histogramů.
   `github.com/HdrHistogram/HdrHistogram`

8. **E. D. Lazowska a kol.: Quantitative System Performance** (1984), kap. 3 o základních zákonech (Littleův zákon, zákon interaktivní doby odezvy). Pro hlubší pochopení, proč se uzavřený systém sám přibrzdí.

TAHÁK NA KARTIČKU

Měření: posílat podle plánu · čas od plánovaného odeslání · kontrolovat propustnost.

Shrnutí: percentily + maximum · nikdy neprůměrovat percentily · slučovat histogramy.

Tři otázky ke grafu: Odkud se počítá čas? Je to průměr? Jak se agregovalo?